

CLASE 1 — Introducción al Backend, Node.js y APIs REST

Duración sugerida

2 a 3 horas

Objetivo General de la Clase

Comprender cómo funciona el backend moderno y crear el primer proyecto utilizando Node.js.

Objetivos de Aprendizaje

Al finalizar la clase el estudiante podrá:

- Comprender qué es el backend
 - Diferenciar frontend y backend
 - Entender el modelo cliente-servidor
 - Comprender qué es una API
 - Entender qué es REST
 - Comprender el formato JSON
 - Instalar Node.js correctamente
 - Verificar instalación de Node y npm
 - Crear el primer proyecto Node.js
-

Parte 1 — Introducción al Desarrollo Backend

¿Qué es el Backend?

El backend es la parte de una aplicación que funciona en el servidor y se encarga de:

- Procesar información
 - Guardar datos
 - Validar usuarios
 - Conectarse con bases de datos
 - Enviar respuestas al frontend
-

Ejemplos cotidianos

Cuando haces login en una página:

Frontend

- Muestra formulario
- Captura usuario y contraseña

Backend

- Verifica credenciales
 - Consulta base de datos
 - Valida acceso
 - Devuelve respuesta
-

Frontend vs Backend

Frontend	Backend
Parte visual	Parte lógica
Interfaz gráfica	Procesamiento
HTML, CSS, JS	Node.js, Java, Python, .NET
Navegador	Servidor

Explicación Visual

Usuario
↓
Frontend
↓
API REST
↓
Backend
↓
Base de Datos

Actividad Inicial

Preguntar a los estudiantes:

- ¿Qué aplicaciones usan diariamente?
 - ¿Qué creen que ocurre cuando presionan “Iniciar sesión”?
 - ¿Dónde creen que se guardan los datos?
-

Parte 2 — Modelo Cliente – Servidor

¿Qué es un Cliente?

Es el dispositivo o aplicación que solicita información.

Ejemplos

- Navegador web
 - Aplicación móvil
 - Postman
 - Aplicación desktop
-

¿Qué es un Servidor?

Es el sistema que recibe solicitudes y responde información.

Funciones

- Procesar peticiones
 - Ejecutar lógica
 - Consultar bases de datos
 - Responder datos
-

Flujo Cliente – Servidor

Cliente solicita información



Servidor procesa petición



Servidor responde datos



Cliente muestra información

Ejemplo Real

Instagram

Cliente

Aplicación móvil

Servidor

Procesa:

- Login
- Fotos
- Likes
- Comentarios

Base de datos

Guarda:

- Usuarios
- Publicaciones
- Mensajes

Parte 3 — ¿Qué es una API?

Definición

API significa:

Application Programming Interface

Es un intermediario que permite que aplicaciones se comuniquen entre sí.

Ejemplo sencillo

Frontend ↔ API ↔ Base de Datos

Analogía de Restaurante

Elemento	Equivalente
----------	-------------

Cliente	Usuario
---------	---------

Mesero	API
--------	-----

Cocina	Backend
--------	---------

Comida	Datos
--------	-------

¿Qué hace una API?

- Recibe solicitudes
 - Procesa información
 - Devuelve respuestas
-

Ejemplo de API

Solicitud

GET /usuarios

Respuesta

```
[
  {
    "nombre": "Juan",
    "edad": 25
  }
]
```

Parte 4 — Introducción a REST

¿Qué es REST?

REST es un estilo de arquitectura para construir APIs.

REST utiliza:

- URLs
 - Métodos HTTP
 - JSON
-

Métodos HTTP Principales

Método	Acción
GET	Obtener
POST	Crear
PUT	Actualizar
DELETE	Eliminar

Ejemplos REST

Obtener usuarios

GET /usuarios

Crear usuario

POST /usuarios

Eliminar usuario

DELETE /usuarios/1

Parte 5 — JSON

¿Qué es JSON?

JSON significa:

JavaScript Object Notation

Es el formato más utilizado para intercambiar datos entre aplicaciones.

Ejemplo JSON

```
{  
  "nombre": "Carlos",  
  "edad": 30,  
  "activo": true  
}
```

Reglas JSON

- Llaves `{ }` para objetos
- Corchetes `[ ]` para listas
- Clave y valor

- Strings entre comillas
-

Comparación Visual

Objeto JavaScript

```
const usuario = {  
  nombre:"Ana"  
}
```

JSON

```
{  
  "nombre":"Ana"  
}
```

Parte 6 — Introducción a Node.js

¿Qué es Node.js?

Node.js es un entorno que permite ejecutar JavaScript fuera del navegador.

¿Para qué sirve Node.js?

- Crear servidores
 - APIs REST
 - Aplicaciones backend
 - Automatización
 - Tiempo real
-

Ventajas de Node.js

- Rápido

- Asíncrono
 - Usa JavaScript
 - Gran comunidad
 - Muchas librerías
-

Parte 7 — Instalación de Node.js

Descarga Oficial

[Node.js Official Website](https://nodejs.org/)

Pasos de Instalación

Windows

1. Descargar instalador LTS
 2. Ejecutar instalador
 3. Siguiente → siguiente → instalar
 4. Reiniciar terminal
-

Verificar Instalación

Abrir terminal y ejecutar:

```
node -v
```

Resultado esperado:

```
v22.x.x
```

Verificar npm

```
npm -v
```

¿Qué es npm?

npm significa:

Node Package Manager

Es el administrador de paquetes de Node.js.

¿Qué permite npm?

- Instalar librerías
 - Administrar dependencias
 - Ejecutar scripts
 - Compartir paquetes
-

Ejemplos de paquetes famosos

Paquete	Función
express	Servidor web
mongoose	MongoDB
cors	Permitir conexiones
dotenv	Variables entorno

Parte 8 — Primer Proyecto Node.js

Crear Carpeta

```
mkdir mi-primer-backend
```

Entrar a la carpeta

`cd mi-primer-backend`

Inicializar Proyecto

`npm init -y`

Explicación

Este comando crea:

`package.json`

¿Qué es package.json?

Archivo que contiene:

- Nombre proyecto
 - Dependencias
 - Scripts
 - Configuración
-

Ejemplo package.json

```
{  
  "name": "mi-primer-backend",  
  "version": "1.0.0"  
}
```

Crear Primer Archivo

Crear:

`app.js`

Primer Código Node.js

```
console.log("Hola Backend");
```

Ejecutar Proyecto

```
node app.js
```

Resultado:

Hola Backend

Explicación del Flujo

Archivo JS



Node.js ejecuta código



Resultado en consola

Actividad Práctica Guiada

Crear proyecto llamado:

curso-api

Debe contener:

- package.json
 - archivo app.js
 - mensaje personalizado
-

Mini Reto

Modificar:

```
console.log("Bienvenidos al curso Backend");
```

Agregar:

- Nombre estudiante
 - Edad
 - Ciudad
-

Errores Comunes

Error	Solución
node no reconocido	Reiniciar terminal
npm no funciona	Reinstalar Node
archivo mal nombrado	Revisar extensión
usar comillas incorrectas	Validar sintaxis

Taller de Clase

Actividad Individual

Objetivo

Instalar Node.js y crear primer proyecto backend.

Debe realizar:

1. Instalar Node.js

Verificar:

```
node -v  
npm -v
```

2. Crear proyecto

```
mkdir backend-clase1  
cd backend-clase1  
npm init -y
```

3. Crear archivo

```
app.js
```

4. Mostrar información en consola

```
console.log("Mi primer backend");  
console.log("Nombre: Juan");  
console.log("Ciudad: Bogotá");
```

Preguntas para Evaluar

1. ¿Qué es backend?
 2. ¿Qué diferencia hay entre frontend y backend?
 3. ¿Qué es una API?
 4. ¿Qué significa REST?
 5. ¿Qué es JSON?
 6. ¿Qué hace Node.js?
 7. ¿Qué es npm?
-

Tarea para la Próxima Clase

Investigar:

- Variables en JavaScript
 - Funciones
 - Arrays
 - Objetos
 - Arrow functions
-

Recursos Recomendados

Documentación Oficial

- [Node.js Documentation](#)
- [MDN JavaScript Guide](#)

Herramientas

- [Visual Studio Code](#)
 - [Postman](#)
-

Resultado Esperado de la Clase

Al finalizar la sesión el estudiante debe:

- Comprender el funcionamiento básico del backend
- Entender el modelo cliente-servidor
- Reconocer qué es una API REST
- Comprender JSON
- Instalar Node.js correctamente
- Crear y ejecutar su primer proyecto Node.js